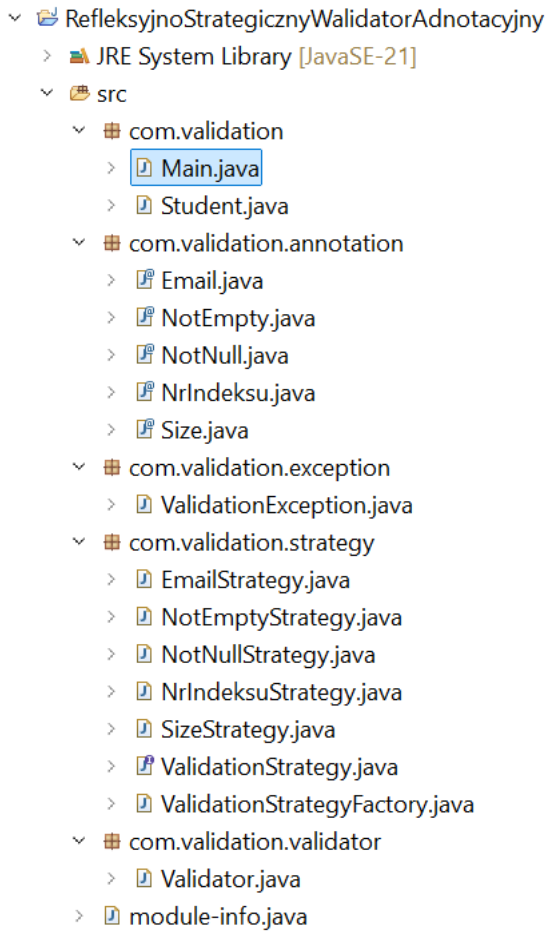
IMPLEMENTACJA WALIDATORA DANYCH Z WYKORZYSTANIEM ADNOTACJI, REFLEKSJI I WZORCA STRATEGIA

1. Opracuj mechanizm walidacji danych wejściowych, który korzysta z refleksji i obsługuje obiekty zawierające pola oznaczone adnotacjami stworzonymi w ramach niniejszego zadania. Celem jest zautomatyzowanie sprawdzania poprawności danych wejściowych bez konieczności pisania dedykowanego kodu walidacyjnego dla każdej klasy. W mechanizmie zastosuj wzorzec projektowy Strategia, aby zarządzać logiką walidacji przypisaną do poszczególnych adnotacji. Dzięki temu rozwiązanie będzie elastyczne i łatwe do rozszerzania o nowe reguły walidacyjne.

Poniżej przedstawiono przykładową strukturę projektu oraz klasę *Student* z polami oznaczonymi odpowiednimi adnotacjami, które należy zdefiniować w ramach zadania.



```java
package com.validation;

import com.validation.annotation.Email;
import com.validation.annotation.NotNull;
import com.validation.annotation.NrIndeksu;
import com.validation.annotation.Size;

public class Student {

	@NotNull(message = "Nie podano imienia")
	@Size(min = 3, max = 50,
		 message = "Imię musi składać się z przynajmniej {min} znaków i nie przekraczać {max}")
	private String imie;

	@NotNull(message = "Nie podano nazwiska")
	@Size(min = 2, max = 50,
	    message = "Nazwisko musi składać się z przynajmniej {min} znaków i nie przekraczać {max}")
	private String nazwisko;

	@NotNull(message = "Nie podano numeru indeksu")
	@NrIndeksu(message = "Niepoprawny numer indeksu") // tylko ośmiocyfrowy łańcuch znaków
	private String nrIndeksu;

	@NotEmpty(message = "Nie podano adresu e-mail")
	@Email(message = "Niepoprawny adres e-mail")
	private String email;

	public Student(){
	}

	public Student(String imie, String nazwisko, String nrIndeksu, String email) {
		this.imie = imie;
		this.nazwisko = nazwisko;
		this.nrIndeksu = nrIndeksu;
		this.email = email;
	}

	//TODO wygeneruj tzw. gettery i setery

}
```

2. W projekcie, pakiecie *com.validation.exception*, utwórz klasę wyjątku *ValidationException*, za którego pomocą będą zwracane informacje o wynikach walidacji. Do obsługi błędów lepiej używać wyjątków zamiast kodów błędów np. w postaci numeru, czy nawet typu wyliczeniowego, ponieważ takie rozwiązanie powoduje bałagan w funkcji wywołującej, która musi zaraz po wykonaniu metody sprawdzać błędy za pomocą dodatkowych warunków. Z tego powodu w przypadku błędu lepiej zgłosić wyjątek. Kod korzystający w wyjątków jest prostszy, czytelniejszy i jego logika nie jest zanieczyszczona obsługą błędów. Takie podejście do tworzenia oprogramowania ułatwi utrzymanie, wszelkie modyfikacje kodu w przyszłości i zmniejsza przy okazji ryzyko popełnienia błędu.

```java
package com.validation.exception;

public class ValidationException extends RuntimeException {
	private static final long serialVersionUID = 1L;

	public ValidationException(String errorMessage) {
		super(errorMessage);
	}

	public ValidationException(String errorMessage, Throwable err) {
		super(errorMessage, err);
	}

}
```

3. Na poniższym listingu przedstawiono przykładową implementację adnotacji *@NotNull*. Wszystkie adnotacje (tj. *@NotNull*, *@NotEmpty*, *@Size*, *@NrIndeksu*, *@Email*) utwórz w pakiecie *com.validation.annotation*.

```
package com.validation.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface NotNull {
      String message() default "Pole wymaga wartości - nie może być null";

}
```

4. Przy tworzeniu strategii realizujących logikę walidacji przypisaną do danej adnotacji wykorzystuj interfejs *ValidationStrategy*. Każda utworzona strategia powinna implementować ten interfejs i znajdować się w pakiecie *com.validation.strategy*.

```
package com.validation.strategy;

import java.lang.reflect.Field;
import java.util.Optional;

public interface ValidationStrategy {

   Optional<String> validate(Field field, Object value);

}
```

Poniżej przedstawiono przykładową klasę *NotNullStrategy* obsługującą adnotację *@NotNull*.

```
package com.validation.strategy;

import java.lang.reflect.Field;
import java.util.Optional;
import com.validation.annotation.NotNull;

public class NotNullStrategy implements ValidationStrategy {

   @Override
   public Optional<String> validate(Field field, Object value) {
      if (field.isAnnotationPresent(NotNull.class) && value == null) {
         NotNull annotation = field.getAnnotation(NotNull.class);
         String errorInfo = String.format("Pole %s: %s", field.getName(), annotation.message());
         return Optional.of(errorInfo);
      }
      return Optional.empty();
   }

}
```

5. Do udostępniania strategii utwórz fabrykę tj. klasę *ValidationStrategyFactory*.

```
package com.validation.strategy;

import java.lang.annotation.Annotation;
import java.util.HashMap;
import java.util.Map;
import com.validation.annotation.NotNull;
import com.validation.annotation.Size;

public class ValidationStrategyFactory {

	// fragment Class<? extends Annotation> określa, że kluczem mapy mogą być
	// tylko klasy, które rozszerzają lub implementują interfejs Annotation
	private static final Map<Class<? extends Annotation>, ValidationStrategy>
	                                                                    strategies = new HashMap<>();

	static {
		strategies.put(NotNull.class, new NotNullStrategy());

		//TODO zarejestruj kolejne strategie ...
	}

	private ValidationStrategyFactory() {
	}

	public static ValidationStrategy getStrategy(Annotation annotation) {
		return strategies.get(annotation.annotationType());
	}
}
```

6. W pakiecie *com.validation.validator* utwórz klasę *Validator* z metodą *validate*, która w przypadku błędów walidacji rzuci wyjątkiem utworzonej wcześniej klasy *ValidationException*.

```
package com.validation.validator;

import java.lang.annotation.Annotation;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import com.validation.exception.ValidationException;
import com.validation.strategy.ValidationStrategy;
import com.validation.strategy.ValidationStrategyFactory;

public class Validator {

   private Validator() {
   }

   public static void validate(Object object) throws ValidationException {
      final List<String> errors = new ArrayList<>();

      final Class<?> clazz = object.getClass();
      // iterowanie po wszystkich polach danej klasy
      for (Field field : clazz.getDeclaredFields()) {
         field.setAccessible(true); // umożliwia dostęp również do pól prywatnych danej klasy
         try {
            Object value = field.get(object);
            // iterowanie po wszystkich adnotacjach danego pola
            for (Annotation annotation : field.getAnnotations()) {

               ValidationStrategy strategy = ValidationStrategyFactory.getStrategy(annotation);
               if (strategy != null) {
                  Optional<String> validationError = strategy.validate(field, value);
                  if(validationError.isPresent()) errors.add(validationError.get());
               }

            }
         } catch (IllegalAccessException e) {
            errors.add("Błąd dostępu do pola: " + field.getName());
         }
      }

      if (!errors.isEmpty()) throw new ValidationException(String.join("\n", errors));
   }
}
```

7. Na koniec w metodzie *main* klasy *Main* przetestuj utworzony mechanizm walidacji.

```
package com.validation;
import com.validation.exception.ValidationException;
import com.validation.validator.Validator;

public class Main {
   public static void main(String[] args) {
      try {
         Student student = new Student();
         student.setEmail("Grzegorz.Brzęczyszczykiewicz#pbs.edu.pl");

        //TODO przetestuj kolejne adnotacje ustawiając błędne wartości

         Validator.validate(student);
      } catch (ValidationException e) {
         System.out.println(e.getMessage());
      }
   }
}
```

8. Zadanie dodatkowe

Celem zadania jest wyeliminowanie potrzeby jawnego deklarowania powiązań klas strategii z odpowiednimi adnotacjami walidacyjnymi.

Na początku zdefiniuj adnotację *ValidationFor*, która pozwoli na oznaczenie klasy realizującej strategię walidacyjną i powiąże ją z dedykowaną adnotacją walidacyjną.

```
package com.validation.annotation;

import java.lang.annotation.Annotation;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface ValidationFor {
   Class<? extends Annotation> value();
}
```

Przykład z oznaczeniem istniejącej klasy strategii za pomocą nowo utworzonej adnotacji *@ValidationFor* przedstawiono poniżej.

```
package com.validation.strategy;

import java.lang.reflect.Field;
import java.util.Optional;
import com.validation.annotation.NotNull;
import com.validation.annotation.ValidationFor;

@ValidationFor(NotNull.class)
public class NotNullStrategy implements ValidationStrategy {
   @Override
   public Optional<String> validate(Field field, Object value) {
      if (field.isAnnotationPresent(NotNull.class) && value == null) {
        NotNull annotation = field.getAnnotation(NotNull.class);
        String errorInfo = String.format("Pole %s: %s", field.getName(), annotation.message());
        return Optional.of(errorInfo);
      }
      return Optional.empty();
   }
}
```

W dalszej kolejności zaimplementuj mechanizm umożliwiający automatyczne wyszukiwanie strategii oraz ich powiązanie z odpowiednimi adnotacjami walidacyjnymi. Rozwiązanie powinno odnajdywać wszystkie klasy oznaczone adnotacją *@ValidationFor*, a następnie na podstawie wartości jej atrybutu *value* uzupełniać mapę *Map<Class<? extends Annotation>, ValidationStrategy> strategies* klasy *ValidationStrategyFactory*.

```
public class ValidationStrategyFactory {
      private static final Map<Class<? extends Annotation>, ValidationStrategy>
                                                                    strategies = new HashMap<>();
      static {
         //strategies.put(NotNull.class, new NotNullStrategy());
         //strategies.put(Size.class, new SizeStrategy());
         //...
         // TODO usuń jawne, umieszczane w mapie powiązania adnotacji walidacyjnych z klasami
         // strategii realizującymi logikę walidacji i zaimplementuj mechanizm automatycznego
         // wyszukiwania strategii i ich wiązania z odpowiednimi adnotacjami walidacyjnymi
         // (bazujący na nowo utworzonej adnotacji @ValidationFor)
      }

        //...
}
```

LITERATURA

- Herbert Schildt, *Java. Kompendium programisty. Wydanie X*, 2018, Helion
- George Reese, *Java. Aplikacje bazodanowe*. *Najlepsze rozwiązania*. 2003, Helion

PRZYDATNE SKRÓTY

CTRL + SHIFT + L – pokazuje wszystkie dostępne skróty

CTRL + SHIFT + F – formatowanie kodu

SHIFT + ALT + R – zmiana nazwy klasy, metody lub zmiennej itp., trzeba wcześniej ustawić kursor na nazwie

SHIFT + ALT + L – utworzenie zmiennej z zaznaczonego fragmentu kodu

SHIFT + ALT + M – utworzenie metody z zaznaczonego fragmentu kodu

CTRL + ALT + *STRZAŁKA W GÓRĘ* – skopiowanie linijki i wklejenie w bieżącym wierszu

CTRL + ALT + *STRZAŁKA W DÓŁ* – skopiowania bieżącej linijki i wklejenie poniżej

CTRL + SHIFT + O – automatyczne dodawanie i porządkowanie sekcji importów

CTRL + 1 – „zrób to co chcę zrobić”, m.in. sugestie rozwiązań bieżącego problemu

CTRL + Q – przejście do miejsca ostatniej modyfikacji

F11 – debugowanie aplikacji

CTRL + F11 – uruchomienie aplikacji

CTRL + M – powiększenie okna

Ustawienie kursora np. na wywołaniu metody, typie zmiennej, klasie importu itp. i wciśnięcie **F3** powoduje przejście do kodu źródłowego wywoływanej metody, klasy zmiennej, klasy importu itd.

Wpisanie ***sysout*** i naciśnięcie skrótu **CTRL + SPACJA** spowoduje wstawienie *System.out.println();*